

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6.4

UPGRADE TO 2.2

 Copy as Markdown

Upgrade to 2.2

Serverpod 2.2 includes new test tools that make it easy to create tests for endpoint methods. For new projects they are configured by default, but existing projects need to go through some steps to enable it (see below). The full documentation of this feature can also be found [here](#).

Add test tools to existing projects

For existing non-Mini projects, a few extra things need to be done:

1. Add the `server_test_tools_path` key with the value `test/integration/test_tools` to `config/generator.yaml`:

```
server_test_tools_path: test/integration/test_tools
```

Without this key, the test tools file is not generated. With the above config the location of the test tools file is `test/integration/test_tools/serverpod_test_tools.dart`, but this can be set to any folder (though should be outside of `lib` as per Dart's test conventions).

2. New projects now come with a test profile in `docker-compose.yaml`. This is not strictly mandatory, but is recommended to ensure that the testing state is never polluted. Add the snippet below to the `docker-compose.yaml` file in the server directory:

```
# Add to the existing services
postgres_test:
  image: postgres:16.3
  ports:
    - '9090:5432'
  environment:
    POSTGRES_USER: postgres
    POSTGRES_DB: <projectname>_test
    POSTGRES_PASSWORD: "<insert database test password>"
```



```

volumes:
  - <projectname>_test_data:/var/lib/postgresql/data
profiles:
  - '' # Default profile
  - test
redis_test:
  image: redis:6.2.6
  ports:
    - '9091:6379'
  command: redis-server --requirepass "<insert redis test password>"
  environment:
    - REDIS_REPLICATION_MODE=master
  profiles:
    - '' # Default profile
    - test
volumes:
  # ...
  <projectname>_test_data:

```

► Or copy the complete file here.

3. Create a `test.yaml` file and add it to the `config` directory:

```

# This is the configuration file for your test environment.
# All ports are set to zero in this file which makes the server find the next available
port.
# This is needed to enable running tests concurrently. To set up your server, you will
# need to add the name of the database you are connecting to and the user name.
# The password for the database is stored in the config/passwords.yaml.

# Configuration for the main API test server.
apiServer:
  port: 0
  publicHost: localhost
  publicPort: 0
  publicScheme: http

# Configuration for the Insights test server.
insightsServer:
  port: 0
  publicHost: localhost
  publicPort: 0
  publicScheme: http

# Configuration for the web test server.
webServer:
  port: 0
  publicHost: localhost
  publicPort: 0
  publicScheme: http

# This is the database setup for your test server.
database:

```

```
host: localhost
port: 9090
name: <projectname>_test
user: postgres

# This is the setup for your Redis test instance.
redis:
  enabled: false
  host: localhost
  port: 9091
```

4. Add this entry to `config/passwords.yaml`

```
test:
  database: '<insert database test password>'
  redis: '<insert redis test password>'
```

5. Add a `dart_test.yaml` file to the `server` directory (next to `pubspec.yaml`) with the following contents:

```
tags:
  integration: {}
```

6. Finally, add the `test` and `serverpod_test` packages as dev dependencies in `pubspec.yaml`:

```
dev_dependencies:
  serverpod_test: <serverpod version> # Should be same version as the `serverpod` package
  test: ^1.24.2
```

That's it, the project setup should be ready to start using the test tools!